Redux-Toolkit – Gestion API

Pourquoi Redux Toolkit ?

Redux classique fonctionne très bien, mais il présente plusieurs difficultés pour un débutant.

1. Beaucoup de code répétitif

En Redux classique, il faut écrire :

- des **actions** à la main
- des action **creators**
- un **reducer** long avec un switch
- gérer **l'immutabilité** (interdiction de modifier directement le state)
- configurer **DevTools** soi-même

Pour un débutant, c'est long, difficile et il se perd rapidement.

2. Plusieurs librairies à installer

Avec **Redux** classique on doit installer :

- **immer** (pour gérer l'immutabilité)
- **redux-devtools-extension**
  etc…

- Pour un débutant c'est trop.

3. Configuration compliquée

Créer un projet **Redux** prenait minimum 4–5 fichiers avant de commencer.

Redux Toolkit résout tout ça :

- **moins de fichiers**
- **moins de code**
- **immutabilité automatique grâce à Mimer intégré**
- **actions générées automatiquement**
- **DevTools activés automatiquement**
- **reducers plus simples**

- C'est pour cela qu'on dit que Redux Toolkit **optimise Redux**.

Installation

npm install @reduxjs/toolkit

npm install react-redux

Les fonctions importantes de Redux Toolkit

configureStore

Fichier : store.js

```
// importer configureStore depuis Redux Toolkit
// configureStore est une fonction qui crée le store Redux plus facilement
import { configureStore } from "@reduxjs/toolkit";
// importer le reducer des tâches depuis le fichier tasksSlice.js
import tasksReducer from "./tasksSlice";
// créer le store
const store = configureStore({
  // le reducer définit comment l'état (state) change
```

```
  // ici, on ajoute notre reducer de tâches sous la clé 'tasks'
  reducer: {
    tasks: tasksReducer
  }
});
// exporter le store pour l'utiliser dans le Provider de React
export default store;
```

Explications :

1. **configureStore** remplace l'ancien **createStore**.
    - **DevTools** activés automatiquement.
2. **tasksReducer** contient tout le code pour gérer les tâches (**addTask**, **deleteTask**, **toggleTask**).
3. La clé **tasks** devient **state.tasks** dans l'application.
4. const **tasks** = **useSelector**(state => state.**tasks**.**tasks**);
5. export default store permet de connecter le store à React via <**Provider**>.

createAction

Fichier : peut être utilisé directement, mais rarement nécessaire avec createSlice

```
import { createAction } from "@reduxjs/toolkit";
// créer une action manuellement
export const increment = createAction("counter/increment");
```

A noter que :

createAction est rarement utilisé, car createSlice génère les actions automatiquement.

createReducer

Exemple basique : compteur

```
import { createReducer } from "@reduxjs/toolkit";
const initialState = { value: 0 };
export const counterReducer = createReducer(initialState, {
  increment: (state) => {
    // Grâce à Immer, on peut modifier le state directement
    state.value++;
  },
});
```

Explication :

- **Pas besoin de return {...state}**
- **Pas besoin de switch**
- **Immer transforme le code en version immutable automatiquement**

createSlice

Fichier : tasksSlice.js

```
import { createSlice } from "@reduxjs/toolkit";
// état initial
const initialState = {
  tasks: [] // tableau qui contiendra les tâches
};
const tasksSlice = createSlice({
  name: "tasks",       // préfixe des actions
```

```
  initialState,        // état initial
  reducers: {
    addTask: (state, action) => {
      // ajouter une tâche (action.payload = nouvelle tâche)
      state.tasks.push(action.payload);
    },

    deleteTask: (state, action) => {
      // supprimer une tâche par id (action.payload = id)
      state.tasks = state.tasks.filter(t => t.id !== action.payload);
    },

    toggleTask: (state, action) => {
      // basculer l'état 'completed' d'une tâche
      const task = state.tasks.find(t => t.id === action.payload);
      if (task) task.completed = !task.completed;
    }
  }
});

// exporter actions générées automatiquement
export const { addTask, deleteTask, toggleTask } = tasksSlice.actions;
// exporter le reducer
export default tasksSlice.reducer;
```

Explication :

- createSlice crée **actions + reducer + state** dans un seul fichier
- Les actions sont nommées automatiquement : tasks/addTask, tasks/deleteTask, tasks/toggleTask
- Grâce à Immer, on peut **modifier le state directement** dans les reducers

Interaction avec une API (pending, fulfilled, rejected)

Fichier : tasksSlice.js (ajouter après reducers)

```
import { createAsyncThunk } from "@reduxjs/toolkit";
// créer un thunk pour charger des utilisateurs depuis une API
export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",      // type d'action généré automatiquement
  async () => {            // fonction qui fait l'appel API
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    return response.json(); // données reçues deviennent action.payload
  }
);
// ajouter les cas async dans extraReducers
const tasksSliceWithAPI = createSlice({
  name: "tasks",
  initialState,
  reducers: { /* ...reducers existants... */ },
  extraReducers: (builder) => {
    builder.addCase(fetchUsers.pending, (state) => {
```

```
    // La requête vient de commencer
    state.loading = true;
  });

  builder.addCase(fetchUsers.fulfilled, (state, action) => {
    // La requête a réussi
    state.loading = false;
    state.users = action.payload; // données reçues
  });

  builder.addCase(fetchUsers.rejected, (state) => {
    // Une erreur s'est produite
    state.loading = false;
    state.error = "Erreur lors du chargement";
  });
 }
});
```

Explications :

- **pending = la requête commence**
- **fulfilled = la requête réussit**
- **rejected = la requête échoue**

Pourquoi Redux Toolkit est une optimisation ?

Avant Redux Toolkit	Avec Redux Toolkit
createStore	configureStore
actions.js	actions auto-générées
reducers avec switch	reducers simples
immutabilité manuelle	Immer automatique
DevTools manuel	DevTools intégrés
beaucoup de fichiers	1 slice

Redux Toolkit simplifie Redux sans changer les concepts de base.

EN Résumé :

Concept	Explication simple
configureStore	Crée le store automatiquement
createAction	Rarement utilisé, génère une action
createReducer	Reducer simple sans switch
createSlice	State + actions + reducers dans un seul fichier
createAsyncThunk	Pour gérer les API (pending, fulfilled, rejected)